TF-IDF Guide: Using scikit-learn for TF-IDF implementation

A practical how-to guide on using the Python library scikit-learn for TF-IDF implementation.

Capital One Tech

September 7, 2023 | 6 min read

Using scikit-learn for term frequency-inverse document frequency in NLP projects

Term frequency-inverse document frequency (TF-IDF) is a popular statistical method that quantifies the importance of words within a document and across a corpus, providing valuable insights for tasks such as information retrieval, document clustering and text classification. It gives users the ability to easily transform raw text data into meaningful numerical representations

that machine learning models can use for sentiment analysis, topic modeling and document similarity detection, among others.

Developers can leverage TF-IDF in their projects with Python libraries like scikit-learn to streamline their workflows and build more accurate and effective solutions to complex language processing challenges.

TF-IDF and scikit-learn

Scikit-learn, also called sklearn, offers a seamless way to effectively utilize the power of TF-IDF in machine learning and natural language processing (NLP) projects through its *TfidfVectorizer* and *TfidfTransformer* classes.

What is scikit-learn?

The scikit-learn open-source Python library offers a comprehensive suite of tools for machine learning and statistical modeling, including classification, regression, clustering and dimensionality reduction — all accessible through a consistent interface. As one of the most popular libraries for ML and NLP practitioners, it provides an uncomplicated implementation of the TF-IDF algorithm through its *TfidfVectorizer* module.

The scikit-learn library was designed with simplicity and ease of use in mind. It has grown into an indispensable resource for data scientists and analysts who are looking to streamline their workflows and build effective solutions to complex language processing challenges.

Explaining TfidfVectorizer and TfidfTransformer classes in scikit-learn

Scikit-learn offers two primary classes for implementing the TF-IDF algorithm: *TfidfVectorizer* and *TfidfTransformer*.

The *TfidfVectorizer* class combines the functionalities of both CountVectorizer, which tokenizes text data into individual words or n-grams while counting their occurrences, and *TfidfTransformer*, which applies the actual TF-IDF transformation. This enables the direct conversion of raw text documents into a matrix of TF-IDF features without having to compute term frequencies separately. It's especially useful when working with large corpora since it simplifies the preprocessing pipeline.

If you already have term frequency counts, such as counts obtained using *CountVectorizer*, you can use the *TfidfTransformer* class to apply the TF-IDF transformation. This class simply computes the IDF values and scales the term frequencies accordingly.

Both the *TfidfVectorizer* and *TfidfTransformer* classes offer various customization options, such as adjusting tokenization patterns, applying stopword removal and modifying IDF smoothing

parameters.

How to implement TF-IDF with scikit-learn

Thanks to the *TfidfVectorizer* class, implementing TF-IDF with scikit-learn is a fairly straightforward process. The first step is importing *TfidfVectorizer* and creating a list of documents to analyze and convert into TF-IDF features:

```
# Import TfidfVectorizer from sklearn.feature_extraction.text
from sklearn.feature_extraction.text import TfidfVectorizer
# Create a list of documents
corpus = [ 'This is the first document.', 'This document is the second
document.', 'And this is the third one.', 'Is this the first document?' ]
```

Next, create an instance of the *TfidfVectorizer* class with the desired customization options, such as tokenization patterns, stopword removal or IDF smoothing parameters. Then, to fit and transform the corpus, call the *fit_transform()* method on the vectorizer instance and pass in the corpus. This computes term frequencies and inverse document frequencies while transforming the text data into a matrix of TF-IDF features:

```
# Create an instance of the TfidfVectorizer class
vectorizer = TfidfVectorizer()
# Pass the corpus to the fit_transform method
X = vectorizer.fit_transform(corpus)
```

Finally, call *get_feature_names()* to inspect feature names and their corresponding TF-IDF values, then convert the variable to an array using *toarray()*:

```
# Inspect feature names and TF-IDF values
print(vectorizer.get_feature_names_out())
# Convert the variable to an array
X.toarray()
```

By following these steps, you can implement TF-IDF with scikit-learn and transform your raw text data into valuable numerical representations for further analysis or feeding into machine learning models.

Real-world TF-IDF applications by Capital One

Data scientists and engineers can apply TF-IDF in their ML projects in numerous ways. Real-world TF-IDF examples are found in applications produced by Capital One. Some of the ways we leverage the power of TF-IDF to enhance our ML projects include:

Virtual card numbers

At Capital One, machine learning techniques are used to enhance the capabilities of Eno, their intelligent assistant. Eno provides a variety of services, such as checking bank balances and offering proactive account monitoring. One unique feature of Eno is its ability to generate virtual card numbers, or VCNs, which are unique credit card numbers for each merchant a customer shops with.

Capital One designed this feature to reduce the exposure of its customers' main credit card numbers to prevent theft and fraud. Other applications of ML techniques, such as those applied with the Eno browser extension, improve this goal by detecting payment pages and locating payment fields more accurately.

Capital One improved the browser extension used for Eno's VCN capabilities by using *TensorFlow* to implement two logistic regression models, one of which leverages TF-IDF.

A payment page classification model detects whether a customer is on a payment page. It uses text and HTML attributes from the page's `*<title>*` and `*<head>*` elements along with nearby based on pixel coordinates as input. These features are then weighted using TF-IDF, a method that considers both the frequency of a term in the document and the number of documents containing the term.

In this scenario, the application of TF-IDF improves accuracy, resulting in better payment page detection and, in turn, making Capital One's VCNs more accurate and less error-prone.

Customer intent prediction

Data scientists and engineers can use TF-IDF to predict customer intent and improve customer satisfaction.

For example, analyzing clickstream data and applying TF-IDF can help predict why a customer might call a service line before they express any concerns. This proactive approach to customer service can reduce call volume and call handling time, elevating the customer service experience.

Engineers at Capital One have applied TF-IDF and other statistical methods to predict customer intent and support Capital One customer service teams.

One specific method of analysis involves correlating the web pages a customer viewed within one hour before placing a customer service call, with the reason for the call inferred by the interactive

voice response system. Understanding which pages a customer frequently visited before calling in helps customer service teams predict the intent of a call more accurately.

TF-IDF is used in combination with *Cosine Similarity* to measure the similarity between the words used in the web page titles and the customer's utterances during their interaction with an interactive voice response system. By understanding the possible reasons for contacting customer service, Capital One support members can be fitted with the right tools and knowledge to provide superior customer service.

Try it yourself with Capital One's wine recommender

We created a wine recommender to demonstrate how a recommendation engine built on AWS's SageMaker platform using *BERT* word embeddings works. Learners can follow the tutorial to build their own wine recommendation engine.

The dataset for this app consists of more than 130,000 wine reviews, which were converted into embeddings using *BERT* and stored in an *AWS S3*. These embeddings serve as the dataset for a Nearest Neighbor search implemented using scikit-learn's estimator.

Those interested in learning more can follow the wine recommendation tutorial and then implement TD-IDF to improve its performance. While the application is simpler, the same approach is effective with larger datasets containing diverse descriptions, so it's a key skill in any data analyst's skill set.

Harnessing the power of scikit-learn for TF-IDF and beyond

Scikit-learn offers an efficient and user-friendly approach to text analysis, supporting the development and deployment of advanced NLP solutions. Its versatility extends beyond TF-IDF, since it provides a comprehensive suite of tools for various NLP tasks, such as sentiment analysis, topic modeling and text classification.

Capital One continues to promote ML and NLP solutions and supports open-source libraries like scikit-learn to power innovative solutions. If you're interested in contributing to this exciting field and making an impact on the future of financial services, consider exploring technology careers at Capital One today.

Capital One Tech

Stories and ideas on development from the people who build it at Capital One.

RELATED CONTENT

MACHINE LEARNING

Improving Virtual Card Numbers with Edge Machine Learning

Article | August 4, 2021

MACHINE LEARNING

Create a Wine Recommender Using NLP on AWS

Article | March 2, 2022

Footnotes

Learn more about FDIC insurance coverage.

DISCLOSURE STATEMENT: © 2023 Capital One. Opinions are those of the individual author. Unless noted otherwise in this post, Capital One is not affiliated with, nor endorsed by, any of the companies mentioned. All trademarks and other intellectual property used or displayed are property of their respective owners. Capital One is not responsible for the content or privacy policies of any linked third-party sites.

Privacy AdChoices Terms & Conditions ©2026 Capital One